Postera: A Fully Quantum-Resistant Privacy Cryptocurrency

Technical Whitepaper v2.0

Postera Development Team
https://postera.network

February 2026

Abstract

We present Postera v2, a cryptocurrency providing strong privacy guarantees that remain secure against both classical and quantum adversaries. Unlike previous versions that used quantum-vulnerable pairing-based proofs for privacy, Postera v2 achieves **full post-quantum security** by combining NIST-standardized signatures (ML-DSA-65, FIPS 204) with hash-based STARK proofs (Plonky2 over the Goldilocks field). This eliminates all reliance on elliptic curve discrete logarithm assumptions, providing complete protection against Shor's algorithm. The protocol maintains a note-based UTXO model similar to Zcash's Sapling but replaces all quantum-vulnerable components with lattice-based and hash-based alternatives. This paper describes the cryptographic foundations, V2 transaction model, migration path from V1, and security properties of the fully quantum-resistant Postera protocol.

Contents

1 Introduction **3**
1.1 The Quantum Threat . . . 3
1.2 Postera v2: Full Quantum Resistance . . . 3

2 Threat Model and Security Goals **3**
2.1 Adversary Capabilities . . . 3
2.2 Security Goals . . . 3

3 Cryptographic Primitives **4**
3.1 Post-Quantum Digital Signatures: ML-DSA-65 . . . 4
3.2 Post-Quantum Zero-Knowledge Proofs: Plonky2 STARKs . . . 4
3.2.1 Plonky2 Parameters . . . 4
3.2.2 Why Plonky2? . . . 4
3.3 Hash Functions . . . 5
3.3.1 Poseidon over Goldilocks . . . 5
3.3.2 Post-Quantum Security of Hash Functions . . . 5

4 V2 Transaction Model **5**
4.1 Notes . . . 5
4.2 V2 Commitments . . . 5
4.3 V2 Nullifiers . . . 6
4.4 V2 Commitment Tree . . . 6
4.5 V2 Transaction Structure . . . 6
4.6 Combined STARK Proof . . . 7
4.7 Dynamic Circuit Building . . . 7

5 V1 to V2 Migration **7**
5.1 Migration Transaction 7
5.2 Migration Incentives 7
5.3 Deprecation Timeline 7

6 Security Analysis **8**
6.1 Full Post-Quantum Security 8
6.2 Comparison with V1 8
6.3 Double-Spend Prevention 8

7 Performance **8**

8 Implementation **9**
8.1 Browser-Based WASM Prover 9
8.2 Server-Side Verification 9

9 Future Work **9**

10 Conclusion **9**

1 Introduction

1.1 The Quantum Threat

The development of practical quantum computers poses an existential threat to current cryptographic systems. Shor's algorithm can solve the discrete logarithm problem in polynomial time, breaking:

- **ECDSA/EdDSA signatures**: Used by Bitcoin, Ethereum, and most cryptocurrencies
- **Pairing-based cryptography**: Used by Groth16 zk-SNARKs (Zcash, Tornado Cash)
- **Pedersen commitments**: Used for hiding transaction amounts

While Postera v1 addressed signature security with ML-DSA-65, the privacy layer remained vulnerable due to its use of BN254-based Groth16 proofs and Pedersen commitments.

1.2 Postera v2: Full Quantum Resistance

Postera v2 eliminates all quantum-vulnerable components:

Component	V1	V2	Quantum Safe
Signatures	ML-DSA-65	ML-DSA-65	✓
ZK Proofs	Groth16/BN254	Plonky2 STARKs	✓
Commitments	Pedersen/BN254	Poseidon/Goldilocks	✓
Merkle Tree	Poseidon/BN254	Poseidon/Goldilocks	✓
Binding Sig	Schnorr/BN254	Included in STARK	✓

Table 1: Comparison of V1 and V2 cryptographic components

2 Threat Model and Security Goals

2.1 Adversary Capabilities

We consider adversaries with the following capabilities:

- **Classical Adversary**: Polynomial-time bounded classical computer with access to all public blockchain data.
- **Quantum Adversary**: Access to a cryptographically-relevant quantum computer capable of running Shor's algorithm (breaks discrete log) and Grover's algorithm (quadratic speedup for search).
- **Network Adversary**: Ability to observe, delay, or reorder network messages.

2.2 Security Goals

Definition 1 (Post-Quantum Transaction Integrity)**.** *No adversary, even with quantum computing capabilities, can create a valid transaction that spends funds they do not own.*

Definition 2 (Post-Quantum Balance Privacy)**.** *No adversary, even with quantum computing capabilities, can determine the balance of any address without knowledge of the corresponding viewing key.*

Definition 3 (Post-Quantum Transaction Privacy)**.** *No adversary, even with quantum computing capabilities, can link the sender and receiver of a transaction, nor determine the transaction amount.*

3 Cryptographic Primitives

3.1 Post-Quantum Digital Signatures: ML-DSA-65

Postera uses ML-DSA-65 (Module-Lattice Digital Signature Algorithm), standardized as FIPS 204 by NIST.

Parameter	Value	Description
Security Level	3 (128-bit)	NIST security category
Public Key Size	1,952 bytes	Verification key
Secret Key Size	4,032 bytes	Signing key
Signature Size	3,309 bytes	Digital signature

Table 2: ML-DSA-65 parameters

The security of ML-DSA relies on the hardness of Module-LWE and Module-SIS problems, which are believed hard for both classical and quantum computers.

3.2 Post-Quantum Zero-Knowledge Proofs: Plonky2 STARKs

Postera v2 replaces Groth16 with Plonky2, a STARK-based proving system that relies only on hash function security.

Definition 4 (STARK)**.** *A Scalable Transparent ARgument of Knowledge is a proof system where:*

- ***Transparency****: No trusted setup required*
- ***Post-Quantum Security****: Security relies on collision-resistant hash functions*
- ***Scalability****: Prover time quasi-linear, verifier time polylogarithmic*

3.2.1 Plonky2 Parameters

Parameter	Value
Field	Goldilocks ($p = 2^{64} - 2^{32} + 1$)
Hash Function	Poseidon (width-12, rate-8)
FRI Parameters	Rate 1/8, 4 query rounds
Security Level	≥ 100 bits post-quantum
Proof Size	≈ 50 KB

Table 3: Plonky2 configuration in Postera v2

3.2.2 Why Plonky2?

1. **Hash-based security**: No reliance on discrete log or pairings
2. **No trusted setup**: Unlike Groth16, no toxic waste
3. **Fast proving**: Optimized for client-side browser execution
4. **Recursive composition**: Enables future protocol upgrades

3.3 Hash Functions

3.3.1 Poseidon over Goldilocks

All in-circuit hashing uses Poseidon optimized for the Goldilocks field:

$$H_{\text{Poseidon}} : \mathbb{F}_p^n \to \mathbb{F}_p^4 \tag{1}$$

where $p = 2^{64} - 2^{32} + 1$ (Goldilocks prime).

The hash outputs 4 field elements (256 bits total), matching Plonky2's internal representation.

Domain	Separator
Note Commitment	1
Value Commitment	2
Nullifier	3
Merkle Empty Leaf	4
Merkle Node	5

Table 4: V2 Poseidon domain separators

3.3.2 Post-Quantum Security of Hash Functions

Grover's algorithm provides a quadratic speedup for brute-force search, reducing effective security:

- 256-bit hash $\to$ 128-bit post-quantum security
- Goldilocks field elements: 4×64 bits $= 256$ bits

This provides adequate post-quantum security margins.

4 V2 Transaction Model

4.1 Notes

The fundamental unit of value remains a **note**:

Definition 5 (V2 Note)**.** *A note $\mathcal{N}$ is a tuple (v, pk_h, r) where:*

- $v \in \{0, 1, \ldots, 2^{64} - 1\}$ *is the value*
- $pk_h \in \mathbb{F}_p^4$ *is the recipient's public key hash (4 Goldilocks elements)*
- $r \in \mathbb{F}_p^4$ *is random blinding factor (4 Goldilocks elements)*

4.2 V2 Commitments

Unlike V1's Pedersen commitments (which hide values using elliptic curve discrete log), V2 uses hash-based commitments:

Definition 6 (V2 Note Commitment)**.** *The commitment to a note $\mathcal{N} = (v, pk_h, r)$ is:*

$$cm = H_{Poseidon}(1, v, pk_h[0..3], r[0..3]) \tag{2}$$

Output: 4 Goldilocks field elements (32 bytes serialized).

Security: The commitment is:

- **Hiding**: Without r, the value cannot be determined (hash preimage resistance)
- **Binding**: Cannot find two different notes with the same commitment (collision resistance)

Both properties hold against quantum adversaries.

4.3 V2 Nullifiers

Definition 7 (V2 Nullifier)**.** *The nullifier for a note with commitment cm at position ρ, using nullifier key nk:*

$$nf = H_{Poseidon}(3, nk[0..3], cm[0..3], \rho) \tag{3}$$

Note: V2 nullifiers include the full commitment (not just position), providing stronger unlinkability.

4.4 V2 Commitment Tree

A Merkle tree of depth 32 using Poseidon/Goldilocks:

- **Leaves**: Note commitments (4 field elements each)
- **Internal nodes**: $H_{\text{Poseidon}}(5, \text{left}[0..3], \text{right}[0..3])$
- **Empty leaf**: $H_{\text{Poseidon}}(4, 0, 0, 0, 0)$

4.5 V2 Transaction Structure

A V2 shielded transaction consolidates all proofs into a single STARK:

```
ShieldedTransactionV2 {
    version: 2,
    spends: [{
        anchor: [u8; 32],       // Merkle root
        nullifier: [u8; 32],    // Spend nullifier
        signature: [u8; 3309], // ML-DSA-65 signature
        public_key: [u8; 1952] // ML-DSA-65 public key
    }],
    outputs: [{
        note_commitment: [u8; 32],
        encrypted_note: { ciphertext, ephemeral_pk }
    }],
    fee: u64,
    transaction_proof: {
        proof_bytes: Vec<u8>,  // Plonky2 STARK (~50KB)
        public_inputs: {
            merkle_roots: [[u8; 32]],
            nullifiers: [[u8; 32]],
            note_commitments: [[u8; 32]],
            fee: u64
        }
    }
}
```

4.6 Combined STARK Proof

The V2 STARK proves all of the following in a single proof:

1. **For each spend**:
   - Knowledge of note preimage (v, pk_h, r)
   - Note commitment exists in tree (Merkle path verification)
   - Nullifier correctly derived from (nk, cm, ρ)
2. **For each output**:
   - Note commitment correctly computed from (v, pk_h, r)
3. **Balance constraint**:

$$\sum_i v_{\text{spend},i} = \sum_j v_{\text{output},j} + \text{fee} \tag{4}$$

This eliminates the need for separate binding signatures—the balance is enforced inside the STARK.

4.7 Dynamic Circuit Building

Unlike V1's fixed circuits, V2 builds circuits dynamically:

- Circuits are parameterized by $(n_{\text{spends}}, n_{\text{outputs}})$
- Maximum: 10 spends $\times$ 4 outputs
- Circuits are cached after first build for reuse
- "Warmup" pre-builds common shapes (1-5 spends $\times$ 1-2 outputs)

5 V1 to V2 Migration

5.1 Migration Transaction

Users with V1 notes can migrate to V2:

1. **Spend V1 notes**: Using legacy Groth16 proofs
2. **Create V2 outputs**: With Poseidon/Goldilocks commitments
3. **Combined proof**: V1 spends + V2 outputs in single transaction

5.2 Migration Incentives

- V2 transactions have lower fees (no binding signature verification)
- V1 will be deprecated after sufficient migration period
- Quantum computers are advancing—migrate before they arrive

5.3 Deprecation Timeline

1. **Phase 1** (Current): Both V1 and V2 supported, V2 default
2. **Phase 2** (6 months): V1 creation disabled, only spending allowed
3. **Phase 3** (12 months): V1 support removed entirely

6 Security Analysis

6.1 Full Post-Quantum Security

Theorem 1 (V2 Quantum Resistance)**.** *Postera V2 transactions are secure against adversaries with access to cryptographically-relevant quantum computers.*

Proof. V2 relies only on:

1. **ML-DSA-65**: Based on Module-LWE/SIS, no known quantum speedup beyond Grover's $O(\sqrt{N})$
2. **Poseidon hash**: Collision resistance reduced to 2^{128} operations with Grover
3. **Plonky2 STARKs**: Security reduces to hash collision resistance

No component relies on discrete log or factoring, which Shor's algorithm breaks. □

6.2 Comparison with V1

Attack	V1 Vulnerable?	V2 Vulnerable?
Forge signatures	No (ML-DSA)	No (ML-DSA)
Forge ZK proofs	Yes (BN254)	No (STARK)
Open commitments	Yes (Pedersen)	No (Poseidon)
Break binding sig	Yes (Schnorr/BN254)	N/A (in STARK)
Link transactions	Yes (if proofs forged)	No

Table 5: Quantum attack vectors

6.3 Double-Spend Prevention

Theorem 2 (Double-Spend Resistance)**.** *A V2 note can only be spent once, even by a quantum adversary.*

Proof. The nullifier $\mathrm{nf} = H(nk, \mathrm{cm}, \rho)$ is deterministic. The STARK proves correct nullifier derivation. Duplicate nullifiers are rejected by consensus. Hash collision would require 2^{128} quantum operations. □

7 Performance

Operation	V1 Time	V2 Time	Notes
Proof generation (1 spend)	3s	2s	Browser WASM
Proof generation (4 spends)	12s	5s	Browser WASM
Proof verification	5ms	50ms	Server-side
Proof size	200 bytes	50 KB	Per transaction
Circuit warmup (5×2)	N/A	30s	One-time

Table 6: Performance comparison

The larger proof size is an acceptable trade-off for quantum resistance. Proof generation is actually *faster* in V2 due to Plonky2's optimized WASM implementation.

8 Implementation

8.1 Browser-Based WASM Prover

The Plonky2 prover is compiled to WebAssembly for client-side execution:

```
// TypeScript interface
const prover = new WasmProver();
await prover.prebuild_circuit(4, 2); // Warmup
const proof = prover.prove(witnessJson);
```

Key features:

- Dynamic circuit building for any transaction shape
- Circuit caching for instant subsequent proofs
- Background warmup with progress tracking
- LocalStorage persistence of warmup state

8.2 Server-Side Verification

Proof verification uses native Rust Plonky2:

```
let circuit = CircuitCache::get_or_build(n_spends, n_outputs);
circuit.verify(proof)?;
```

The circuit cache uses thread-safe `Arc<RwLock>` for concurrent access.

9 Future Work

1. **Recursive Proofs**: Aggregate multiple transactions into single proof
2. **Light Clients**: STARK-based SPV proofs for mobile
3. **Proof Compression**: Reduce 50KB proofs using recursive SNARKs
4. **Post-Quantum Key Exchange**: Replace ChaCha20-Poly1305 key derivation with Kyber

10 Conclusion

Postera v2 achieves **complete post-quantum security** for private cryptocurrency transactions. By replacing all elliptic curve-based components with hash-based alternatives:

- **Signatures**: ML-DSA-65 (lattice-based, NIST standardized)
- **ZK Proofs**: Plonky2 STARKs (hash-based, no trusted setup)
- **Commitments**: Poseidon/Goldilocks (hash-based)

Users can transact privately today with confidence that their transactions will remain private and secure even after the advent of large-scale quantum computers.

Acknowledgments

We thank the developers of Plonky2 (Polygon Zero), the NIST PQC standardization team, and the Zcash protocol designers whose work inspired Postera's privacy model.

References

[1] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. 2024.

[2] Polygon Zero Team. *Plonky2: Fast Recursive Arguments with PLONK and FRI*. 2022.

[3] Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. IACR Cryptology ePrint Archive, 2018.

[4] Hamburg, M. *Ed448-Goldilocks, a New Elliptic Curve*. IACR Cryptology ePrint Archive, 2015.

[5] Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. USENIX Security 2021.

[6] Hopwood, D., Bowe, S., Hornby, T., Wilcox, N. *Zcash Protocol Specification*. 2023.

[7] Shor, P. *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*. FOCS 1994.

[8] Grover, L. *A Fast Quantum Mechanical Algorithm for Database Search*. STOC 1996.